



Nombre: Sebastian Garcia Rios, Adrian Jaret Rendon Rios

Materia: Topicos de IA

Carrera: Ingenieria en sistemas computacionales

Profesor: Zuriel Dathan Mora Felix

Tarea: Tarea de validación Modulo 3

Unidad: 3

Tarea de Validación III Algoritmo Genético para el Problema del Agente Viajante (TSP)

Objetivo del Proyecto

El propósito de esta actividad fue reconstruir y depurar un código de Algoritmo Genético (AG) proporcionado en teams. El algoritmo está diseñado para resolver el Problema del Agente Viajante (TSP), encontrando la ruta más corta que conecta un conjunto de 7 ciudades (municipios) definidas por coordenadas (X, Y).

La validación implicó identificar y corregir dos errores principales en el código original:

Error de Datos: El *dataset* de ciudades estaba mal definido, causando un `AttributeError` al intentar ejecutar el script.

Error Lógico: El algoritmo no mejoraba (la distancia inicial era igual a la final) debido a un fallo en la preservación del elitismo durante la fase de mutación.



Descripción Breve del Algoritmo Genético

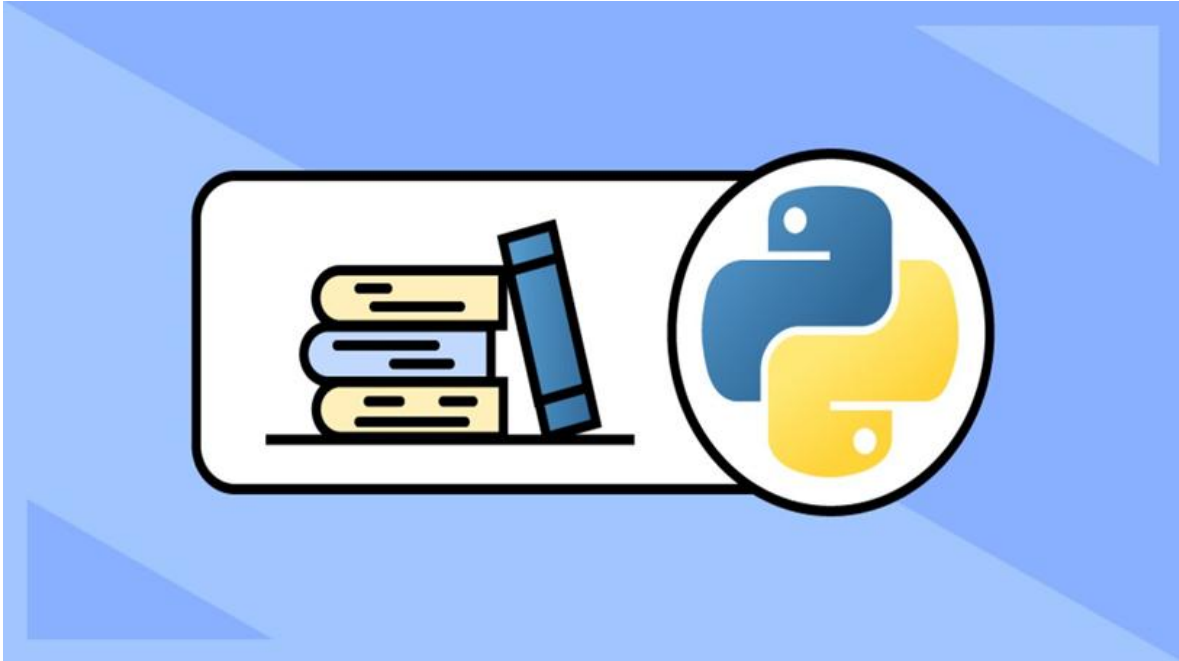
Un Algoritmo Genético es una técnica de optimización inspirada en la selección natural. Funciona siguiendo estos pasos:

1. **Población Inicial:** Se crea un conjunto de soluciones aleatorias (100 rutas de ciudades).
2. **Evaluación (Aptitud):** A cada ruta se le asigna un puntaje. En este problema, la aptitud es $1 / \text{distancia total}$. Una distancia más corta significa una aptitud más alta.
3. **Selección:** Se seleccionan las mejores rutas (padres). Este código usa **Elitismo** (los 20 mejores pasan directo) y **Selección por Ruleta** (los 80 restantes se eligen al azar, dando prioridad a los más aptos). Ambos ejemplos ya vistos y practicados en clase.
4. **Cruce (Reproducción):** Los padres se combinan para crear "hijos" (nuevas rutas).
5. **Mutación:** Se aplica un pequeño cambio aleatorio (intercambiar dos ciudades) a las rutas "hijo" (excepto a la élite) para mantener la diversidad genética.
6. **Evolución:** El ciclo se repite 500 veces (generaciones), mejorando la solución en cada ciclo.

Dependencias de Software (Bibliotecas)

Para ejecutar este script, se requiere Python 3 y las siguientes bibliotecas externas.

- numpy: Utilizada para cálculos numéricos rápidos (Teorema de Pitágoras).
- pandas: Utilizada para facilitar la implementación de la "Selección por Ruleta".



Resultados de la Validación

La ejecución del script corregido demuestra que los errores fueron solucionados y que el algoritmo ahora converge a una solución óptima (mejora).

Prueba de Ejecución (Salida de la Terminal):

Iniciando Algoritmo Genético para el Problema del Viajante...

Distancia Inicial (Mejor ruta): 35.3761335900526

Distancia Final (Mejor ruta): 31.671294629138898

Tiempo total de ejecución: 63.40 segundos

--- ¡Proceso completado! ---

La mejor ruta encontrada visita las ciudades en este orden:

INICIO -> (11,6) -> (7,1) -> (3,3) -> (5,7) -> (1,9) -> (5,10) -> (9,9) -> FIN (Regreso a (11,6))

Conclusión de la Validación: La Distancia Final (31.67) es significativamente menor que la Distancia Inicial (35.37), lo que comprueba que el bug lógico fue corregido y que el elitismo funciona, permitiendo al algoritmo mejorar la solución a lo largo de las 500 generaciones.